

Sprint Documentation

Sprint 15	Start Date: 08/04/2026	Final Date: 13/04/2026
Projetc Name:	civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
João Pedro Chapichi		
João Antonio Gomes Cestonaro		

Sprint Backlog

Task #	Description	Start Date	Final date
#75	Adicionar relógio referente a instuição e adicionar cadastro de orçamento por kilowatt	08/04/2026	13/04/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#75	Modificar os dados aceitos em TipoDespesa, Despesa para aceitar diversos tipos de despesa e tornar o sistema escalável. Criar uma nova tabela TipoCodigo para relacionar o codigo em despesa com as respectivas despesas.	João Pedro Chapichi, João Antonio Gomes Cestonaro	Completo	10 horas	10 horas

Sprint Description

Tarefa: Sprint 15 - Adicionar relógio referente a instuição e adicionar cadastro de orçamento por kilowatt

Objetivo

Modificar os dados aceitos em TipoDespesa, Despesa para aceitar diversos tipos de despesa e tornar o sistema escalável. Criar uma nova tabela TipoCodigo para relacionar o codigo em despesa com as respectivas despesas.

Critérios de Aceite

- Tabela TipoCodigo com os devidos atributos.
- Modificações na tabela Despesa.
- Modificações na tabela TipoDespesa.
- Relacionamentos entre tabelas corretos.
- Adicionar cadastro de Kilowatt na despesa.

Escopo da Implementação

Alterações no Modelo:

```
```csharp id="o1n7s2"
```

```
public Int Id { get; set; }
```

```
public Int Nome { get; set; }
```

```
public Int Descricao { get; set; }
```

```
```
```

Observações

- Sem observações

Sprint Results

Sobre os Endpoints

- GET listando todos os gastos de uma determinada instituição (soma de todas as despesas da instituição).
Retornando ID da instituição, nome das despesas, e total de gastos.
- GET listando o orçamento total disponível para uma determinada instituição (orçamento – total de gastos = orçamento disponível).
- Retornando ID da instituição, nome da instituição e orçamento total.
- GET listando todos os gastos de uma determinada secretaria (soma de todas as despesas de todas as instituições dessa secretaria).
- Retornando ID da secretaria, nome da secretaria, quantidade de instituições, quantidade de despesas e o total de gastos.
- GET listando o orçamento disponível para uma determinada secretaria (orçamento total da secretaria – gastos totais da secretaria).
- Retornando ID da secretaria, nome da secretaria, quantidade de instituições e valor total de orçamento disponível.

Sobre o Relógio:

- Criando a tabela TipoCodigo (id – autoincrementado, nome e descrição).
- A tabela TipoDespesa recebe o idTipoCodigo.
- A tabela Despesa recebe o idTipoDespesa.
- Na tabela despesa foi adicionado o campo código para registrar qualquer código necessário para a despesa.

```
Services > Interfaces > ITipoCodigoService.cs
1 using Civitas.WebAPI.Objects.Dtos.Entities;
2 using Civitas.WebAPI.Objects.Models;
3
4 namespace Civitas.WebAPI.Services.Interfaces
5 {
6     public interface ITipoCodigoService : IGenericService<TipoCodigo, TipoCodigoDTO>
7     {
8     }
9 }
10
```

```
Services > Entities > TipoCodigoService.cs
1 using AutoMapper;
2 using Civitas.WebAPI.Data.Interfaces;
3 using Civitas.WebAPI.Objects.Dtos.Entities;
4 using Civitas.WebAPI.Objects.Models;
5 using Civitas.WebAPI.Services.Interfaces;
6
7 namespace Civitas.WebAPI.Services.Entities
8 {
9     public class TipoCodigoService : GenericService<TipoCodigo, TipoCodigoDTO>, ITipoCodigoService
10    {
11        private readonly ITipoCodigoRepository _tipoCodigoRepository;
12        private readonly IMapper _mapper;
13
14        public TipoCodigoService(ITipoCodigoRepository tipoCodigoRepository, IMapper mapper)
15            : base(tipoCodigoRepository, mapper)
16        {
17            _tipoCodigoRepository = tipoCodigoRepository;
18            _mapper = mapper;
19        }
20    }
21 }
22
```

```
Objects > Dtos > Entities > TipoCodigoDTO.cs
1 namespace Civitas.WebAPI.Objects.Dtos.Entities
2 {
3     public class TipoCodigoDTO
4     {
5         public int Id { get; set; }
6         public string Nome { get; set; }
7         public string Descricao { get; set; }
8     }
9 }
10
```

Data > Interfaces > ITipoCodigoRepository.cs

```
1 using Civitas.WebAPI.Objects.Models;
2
3 namespace Civitas.WebAPI.Data.Interfaces
4 {
5     public interface ITipoCodigoRepository : IGenericRepository<TipoCodigo>
6     {
7
8     }
9 }
10
```

Data > Repositories > TipoCodigoRepository.cs

```
1 using Civitas.WebAPI.Data.Interfaces;
2 using Civitas.WebAPI.Objects.Models;
3
4 namespace Civitas.WebAPI.Data.Repositories
5 {
6     public class TipoCodigoRepository : GenericRepository<TipoCodigo>, ITipoCodigoRepository
7     {
8         private readonly AppDbContext _context;
9
10        public TipoCodigoRepository(AppDbContext context) : base(context)
11        {
12            _context = context;
13        }
14    }
15 }
```

Data > Builders > TipoCodigoBuilder.cs

```
1 using Civitas.WebAPI.Objects.Models;
2 using Microsoft.EntityFrameworkCore;
3
4 namespace Civitas.WebAPI.Data.Builders
5 {
6     public class TipoCodigoBuilder
7     {
8         public static void Build(ModelBuilder modelBuilder)
9         {
10             modelBuilder.Entity<TipoCodigo>().HasKey(f => f.Id);
11             modelBuilder.Entity<TipoCodigo>().Property(f => f.Nome).IsRequired();
12             modelBuilder.Entity<TipoCodigo>().Property(f => f.Descricao).IsRequired();
13         }
14     }
15 }
16
```

Controllers > TipoCodigoController.cs

```
1  using Civitas.WebAPI.Objects.Contracts;
2  using Civitas.WebAPI.Objects.Dtos.Entities;
3  using Civitas.WebAPI.Objects.Models;
4  using Civitas.WebAPI.Services.Interfaces;
5  using Microsoft.AspNetCore.Authorization;
6  using Microsoft.AspNetCore.Mvc;
7
8  namespace Civitas.WebAPI.Controllers
9  {
10     /// <summary>
11     /// Controller responsável pelo gerenciamento dos Tipos de Código.
12     /// </summary>
13     [Authorize]
14     [Route("api/tipo-codigo")]
15     [ApiController]
16     public class TipoCodigoController : ControllerBase
17     {
18         private readonly ITipoCodigoService _tipoCodigoService;
19         private readonly Response _response;
20
21         public TipoCodigoController(ITipoCodigoService tipoCodigoService)
22         {
23             _tipoCodigoService = tipoCodigoService;
24             _response = new Response();
25         }
26
27         /// <summary>
28         /// Lista todos os tipos de código.
29         /// </summary>
30         [HttpGet]
31         public async Task<IActionResult> GetAll()
32         {
33             var result = await _tipoCodigoService.GetAll();
34
35             _response.Code = ResponseEnum.SUCCESS;
36             _response.Data = result;
37             _response.Message = "Tipos de código listados com sucesso";
38
39             return Ok(_response);
40         }
41     }
42 }
```

```
0 }
1
2 /// <summary>
3 /// Busca um tipo de código pelo ID.
4 /// </summary>
5 [HttpGet("{id}")]
6 public async Task<IActionResult> GetById(int id)
7 {
8     var result = await _tipoCodigoService.GetById(id);
9
10    if (result == null)
11    {
12        _response.Code = ResponseEnum.NOT_FOUND;
13        _response.Data = null;
14        _response.Message = "Tipo de código não encontrado";
15
16        return NotFound(_response);
17    }
18
19    _response.Code = ResponseEnum.SUCCESS;
20    _response.Data = result;
21    _response.Message = "Tipo de código encontrado com sucesso";
22
23    return Ok(_response);
24 }
25
26 /// <summary>
27 /// Cadastra um novo tipo de código.
28 /// </summary>
29 [HttpPost]
30 public async Task<IActionResult> Post(TipoCodigoDTO dto)
31 {
32     if (dto == null)
33     {
34         _response.Code = ResponseEnum.INVALID;
35         _response.Data = null;
36         _response.Message = "Dados inválidos";
```

```

77
78     return BadRequest(_response);
79 }
80
81 try
82 {
83     await _tipoCodigoService.Create(dto);
84
85     _response.Code = ResponseEnum.SUCCESS;
86     _response.Data = dto;
87     _response.Message = "Tipo de código cadastrado com sucesso";
88
89     return Ok(_response);
90 }
91 catch (Exception ex)
92 {
93     _response.Code = ResponseEnum.ERROR;
94     _response.Message = "Erro ao cadastrar tipo de código";
95     _response.Data = new
96     {
97         ErrorMessage = ex.Message,
98         StackTrace = ex.StackTrace ?? "Sem stack trace disponível"
99     };
100
101     return StatusCode(StatusCodes.Status500InternalServerError, _response);
102 }
103 }
104
105 /// <summary>
106 /// Atualiza um tipo de código existente.
107 /// </summary>
108 [HttpPut("{id}")]
109 public async Task<IActionResult> Put(int id, TipoCodigoDTO dto)
110 {

```

```

110 {
111     if (dto == null)
112     {
113         _response.Code = ResponseEnum.INVALID;
114         _response.Data = null;
115         _response.Message = "Dados inválidos";
116
117         return BadRequest(_response);
118     }
119
120     try
121     {
122         var existing = await _tipoCodigoService.GetById(id);
123         if (existing == null)
124         {
125             _response.Code = ResponseEnum.NOT_FOUND;
126             _response.Data = null;
127             _response.Message = "Tipo de código não encontrado";
128
129             return NotFound(_response);
130         }
131
132         await _tipoCodigoService.Update(dto, id);
133
134         _response.Code = ResponseEnum.SUCCESS;
135         _response.Data = dto;
136         _response.Message = "Tipo de código atualizado com sucesso";
137
138         return Ok(_response);
139     }
140     catch (Exception ex)
141     {
142         _response.Code = ResponseEnum.ERROR;
143         _response.Message = "Erro ao atualizar tipo de código";
144         _response.Data = new

```

```

146         ErrorMessage = ex.Message,
147         StackTrace = ex.StackTrace ?? "Sem stack trace disponível"
148     };
149
150     return StatusCode(StatusCodes.Status500InternalServerError, _response);
151 }
152 }
153
154 /// <summary>
155 /// Remove um tipo de código.
156 /// </summary>
157 [HttpDelete("{id}")]
158 public async Task<IActionResult> Delete(int id)
159 {
160     try
161     {
162         var existing = await _tipoCodigoService.GetById(id);
163         if (existing == null)
164         {
165             _response.Code = ResponseEnum.NOT_FOUND;
166             _response.Data = null;
167             _response.Message = "Tipo de código não encontrado";
168
169             return NotFound(_response);
170         }
171
172         await _tipoCodigoService.Remove(id);
173
174         _response.Code = ResponseEnum.SUCCESS;
175         _response.Data = null;
176         _response.Message = "Tipo de código removido com sucesso";
177
178         return Ok(_response);
179     }
180     catch (Exception ex)
181     {
182         _response.Code = ResponseEnum.ERROR;
183         _response.Message = "Erro ao remover tipo de código";
184         _response.Data = new
185         {
186             ErrorMessage = ex.Message,
187             StackTrace = ex.StackTrace ?? "Sem stack trace disponível"
188         };
189
190         return StatusCode(StatusCodes.Status500InternalServerError, _response);
191     }
192 }
193 }
194 }

```

```

public async Task<InstituicaoGastosDTO> GetGastosByInstituicaoIdAsync(int instituicaoId)
{
    return await _instituicaoRepository.GetGastosByInstituicaoIdAsync(instituicaoId);
}

public async Task<InstituicaoOrcamentoDisponivelDTO> GetOrcamentoDisponivelByInstituicaoIdAsync(int instituicaoId)
{
    return await _instituicaoRepository.GetOrcamentoDisponivelByInstituicaoIdAsync(instituicaoId);
}

```

```

Task<InstituicaoGastosDTO> GetGastosByInstituicaoIdAsync(int instituicaoId);
Task<InstituicaoOrcamentoDisponivelDTO> GetOrcamentoDisponivelByInstituicaoIdAsync(int instituicaoId);
}

```



```

244     public async Task<SecretariaGastosDTO?> GetGastosBySecretariaIdAsync(int secretariaId)
245     {
246         return await _secretariaRepository.GetGastosBySecretariaIdAsync(secretariaId);
247     }
248
249     public async Task<SecretariaOrcamentoDisponivelDTO?> GetOrcamentoDisponivelBySecretariaIdAsync(int secretariaId)
250     {
251         return await _secretariaRepository.GetOrcamentoDisponivelBySecretariaIdAsync(secretariaId);
252     }
253 }

```

```

7 Task ValidarCadastroAsync(SecretariaDTO entityDTO, int? id = null);
8 Task<SecretariaGastosDTO?> GetGastosBySecretariaIdAsync(int secretariaId);
9 Task<SecretariaOrcamentoDisponivelDTO?> GetOrcamentoDisponivelBySecretariaIdAsync(int secretariaId);
10 }

```

Objects > Dtos > Entities > C# InstituicaoGastosDTO.cs

```

1 namespace Civitas.WebAPI.Objects.Dtos.Entities
2 {
3     public class InstituicaoGastosDTO
4     {
5         public int IdInstituicao { get; set; }
6         public string NomeInstituicao { get; set; }
7         public int QuantidadeDespesas { get; set; }
8         public double TotalGastos { get; set; }
9     }
10 }

```

Objects > Dtos > Entities > C# InstituicaoOrcamentoDisponivelDTO.cs

```

1 namespace Civitas.WebAPI.Objects.Dtos.Entities
2 {
3     public class InstituicaoOrcamentoDisponivelDTO
4     {
5         public int IdInstituicao { get; set; }
6         public string NomeInstituicao { get; set; }
7         public double TotalOrcamentoDisponivel { get; set; }
8     }
9 }
10

```

Objects > Dtos > Entities > C# SecretariaGastosDTO.cs

```

1 namespace Civitas.WebAPI.Objects.Dtos.Entities
2 {
3     public class SecretariaGastosDTO
4     {
5         public int IdSecretaria { get; set; }
6         public string NomeSecretaria { get; set; }
7         public int QuantidadeInstituicoes { get; set; }
8         public int QuantidadeDespesas { get; set; }
9         public double TotalGastos { get; set; }
10    }
11 }
12

```

Objects > Dtos > Entities > * SecretariaOrcamentoDisponivelDTO.cs

```
1 namespace Civitas.WebAPI.Objects.Dtos.Entities
2 {
3     public class SecretariaOrcamentoDisponivelDTO
4     {
5         public int IdSecretaria { get; set; }
6         public string NomeSecretaria { get; set; }
7         public int QuantidadeInstituicoes { get; set; }
8         public decimal TotalOrcamentoDisponivel { get; set; }
9     }
10 }
11
```

```
public async Task<InstituicaoGastosDTO?> GetGastosByInstituicaoIdAsync(int instituicaoId)
{
    return await _context.Instituicoes
        .AsNoTracking()
        .Where(instituicao => instituicao.Id == instituicaoId)
        .Select(instituicao => new InstituicaoGastosDTO
        {
            IdInstituicao = instituicao.Id,
            NomeInstituicao = instituicao.Nome,
            QuantidadeDespesas = instituicao.Despesas.Count(despesa => despesa.Situacao == Situacao.ATIVO),
            TotalGastos = instituicao.Despesas
                .Where(despesa => despesa.Situacao == Situacao.ATIVO)
                .Sum(despesa => despesa.ConsumoPrevisto)
        })
        .FirstOrDefaultAsync();
}

public async Task<InstituicaoOrcamentoDisponivelDTO?> GetOrcamentoDisponivelByInstituicaoIdAsync(int instituicaoId)
{
    return await _context.Instituicoes
        .AsNoTracking()
        .Where(instituicao => instituicao.Id == instituicaoId)
        .Select(instituicao => new InstituicaoOrcamentoDisponivelDTO
        {
            IdInstituicao = instituicao.Id,
            NomeInstituicao = instituicao.Nome,
            TotalOrcamentoDisponivel = (instituicao.Orcamento.Sum(orcamento => orcamento.ValorOrcamento) - instituicao.Despesas
                .Where(despesa => despesa.Situacao == Situacao.ATIVO)
                .Sum(despesa => despesa.ConsumoPrevisto))
        })
        .FirstOrDefaultAsync();
}
}
```

```

48 public async Task<SecretariaGastosDTO?> GetGastosBySecretariaIdAsync(int secretariaId)
49 {
50     return await _appDbContext.Secretarias
51         .AsNoTracking()
52         .Where(secretaria => secretaria.IdSecretaria == secretariaId)
53         .Select(secretaria => new SecretariaGastosDTO
54         {
55             IdSecretaria = secretaria.IdSecretaria,
56             NomeSecretaria = secretaria.Nome,
57             QuantidadeInstituicoes = _appDbContext.Instituicoes.Count(instituicao => instituicao.IdSecretaria == secretaria.IdSecretaria),
58             QuantidadeDespesas = _appDbContext.Despesas.Count(despesa => despesa.Instituicao.IdSecretaria == secretaria.IdSecretaria && despesa.Situacao == Situacao.A
59             TotalGastos = _appDbContext.Despesas
60                 .Where(despesa => despesa.Instituicao.IdSecretaria == secretaria.IdSecretaria && despesa.Situacao == Situacao.ATIVO)
61                 .Sum(despesa => despesa.ConsumoPrevisto)
62         })
63         .FirstOrDefaultAsync();
64 }
65
66 public async Task<SecretariaOrcamentoDisponivelDTO?> GetOrcamentoDisponivelBySecretariaIdAsync(int secretariaId)
67 {
68     return await _appDbContext.Secretarias
69         .AsNoTracking()
70         .Where(secretaria => secretaria.IdSecretaria == secretariaId)
71         .Select(secretaria => new SecretariaOrcamentoDisponivelDTO
72         {
73             IdSecretaria = secretaria.IdSecretaria,
74             NomeSecretaria = secretaria.Nome,
75             QuantidadeInstituicoes = _appDbContext.Instituicoes.Count(instituicao => instituicao.IdSecretaria == secretaria.IdSecretaria),
76             TotalOrcamentoDisponivel =
77             (
78                 _appDbContext.Orcamentos
79                 .Where(orçamento => orçamento.Instituicao.IdSecretaria == secretaria.IdSecretaria)
80                 .Sum(orçamento => (decimal?)orçamento.ValorOrcamento) ?? 0
81             )
82             -
83             (
84                 _appDbContext.Despesas
85                 .Where(despesa => despesa.Instituicao.IdSecretaria == secretaria.IdSecretaria && despesa.Situacao == Situacao.ATIVO)
86                 .Sum(despesa => (decimal?)despesa.ConsumoPrevisto) ?? 0
87             )
88         })
89         .FirstOrDefaultAsync();

```

```

Task<SecretariaGastosDTO?> GetGastosBySecretariaIdAsync(int secretariaId);
Task<SecretariaOrcamentoDisponivelDTO?> GetOrcamentoDisponivelBySecretariaIdAsync(int secretariaId);
}

```

```

9 Task<InstituicaoGastosDTO?> GetGastosByInstituicaoIdAsync(int instituicaoId);
10 Task<InstituicaoOrcamentoDisponivelDTO?> GetOrcamentoDisponivelByInstituicaoIdAsync(int instituicaoId);
11

```

```
[HttpGet("{id}/gastos")]
public async Task<IActionResult> GetGastosByInstituicao(int id)
{
    var gastosInstituicao = await _instituicaoService.GetGastosByInstituicaoIdAsync(id);
    if (gastosInstituicao is null)
    {
        _response.Code = ResponseEnum.NOT_FOUND;
        _response.Data = null;
        _response.Message = "Instituição não encontrada";
        return NotFound(_response);
    }

    _response.Code = ResponseEnum.SUCCESS;
    _response.Data = gastosInstituicao;
    _response.Message = "Gastos da instituição obtidos com sucesso";
    return Ok(_response);
}

[HttpGet("{id}/orcamento-disponivel")]
public async Task<IActionResult> GetOrcamentoDisponivelByInstituicao(int id)
{
    var orcamentoInstituicao = await _instituicaoService.GetOrcamentoDisponivelByInstituicaoIdAsync(id);
    if (orcamentoInstituicao is null)
    {
        _response.Code = ResponseEnum.NOT_FOUND;
        _response.Data = null;
        _response.Message = "Instituição não encontrada";
        return NotFound(_response);
    }

    _response.Code = ResponseEnum.SUCCESS;
    _response.Data = orcamentoInstituicao;
    _response.Message = "Orçamento disponível da instituição obtido com sucesso";
    return Ok(_response);
}
```

```

    public async Task<IActionResult> GetGastosBySecretaria(int id)
    {
        try
        {
            var gastosSecretaria = await _secretariaService.GetGastosBySecretariaIdAsync(id);
            if (gastosSecretaria is null)
            {
                _response.Code = ResponseEnum.NOT_FOUND;
                _response.Data = null;
                _response.Message = "Secretaria não encontrada";
                return NotFound(_response);
            }

            _response.Code = ResponseEnum.SUCCESS;
            _response.Data = gastosSecretaria;
            _response.Message = "Gastos da secretaria obtidos com sucesso";
            return Ok(_response);
        }
        catch (Exception ex)
        {
            _response.Code = ResponseEnum.ERROR;
            _response.Message = "Ocorreu um erro ao obter os gastos da secretaria";
            _response.Data = new
            {
                ErrorMessage = ex.Message,
                StackTrace = ex.StackTrace ?? "Sem stack trace disponível"
            };
            return StatusCode(StatusCodes.Status500InternalServerError, _response);
        }
    }

    [HttpGet("{id}/orcamento-disponivel")]
    public async Task<IActionResult> GetOrcamentoDisponivelBySecretaria(int id)
    {
        try
        {
            var orcamentoSecretaria = await _secretariaService.GetOrcamentoDisponivelBySecretariaIdAsync(id);
            if (orcamentoSecretaria is null)
            {
                _response.Code = ResponseEnum.NOT_FOUND;
                _response.Data = null;
                _response.Message = "Secretaria não encontrada";
                return NotFound(_response);
            }

            _response.Code = ResponseEnum.SUCCESS;
            _response.Data = orcamentoSecretaria;
            _response.Message = "Orçamento disponível da secretaria obtido com sucesso";
            return Ok(_response);
        }
        catch (Exception ex)
        {
            _response.Code = ResponseEnum.ERROR;
            _response.Message = "Ocorreu um erro ao obter o orçamento disponível da secretaria";
            _response.Data = new
            {
                ErrorMessage = ex.Message,
                StackTrace = ex.StackTrace ?? "Sem stack trace disponível"
            };
            return StatusCode(StatusCodes.Status500InternalServerError, _response);
        }
    }
}

```

Assinaturas